

CS 301 Midterm — S001 — Question 1

QuickSort has an average case time complexity of $O(n \log n)$.

This comes from the recurrence $T(n) = 2T(n/2) + O(n)$, where the partition step takes $O(n)$ a

In the worst case, when the pivot is always the smallest or largest element (e.g. sorted input), complexity degrades to $O(n^2)$.

CS 301 Midterm — S001 — Question 2

BFS uses a queue and explores level by level.

It finds the shortest path in unweighted graphs.

DFS uses a stack (or recursion) and goes deep first.

DFS is useful for topological sort and cycle detection.

Both have time complexity $O(V+E)$.

CS 301 Midterm — S002 — Question 1

QuickSort divides the array using a pivot.

Average is $O(n \log n)$ but the worst case $O(n^2)$ occurs when pivots are poorly chosen.

The partition step is linear $O(n)$.

CS 301 Midterm — S002 — Question 2

BFS goes broad first, uses a queue.

Good for finding shortest path.

DFS goes deep first, uses stack.

Useful for cycle detection.

Both run in $O(V + E)$ time.